

Networking: Chapter 5

Questions and Solutions

1. 5.1 Introduction to the Control Plane

Question 1: Per-Router Control (*Monolithic*)

What is meant by a control plane that is based on **per-router control**? In such cases, when we say the network control and data planes are implemented “**monolithically**,” what do we mean?

Solution

Per-router control implies a distributed architecture where:

- A **routing algorithm** runs in each and every router.
- Both **forwarding** and **routing** functions are constrained within each individual router.
- Each router has a routing component that communicates with others to compute local forwarding table values.

The term “**monolithic**” is used because each router works as an **independent entity**, implementing its own control and data planes internally. The two planes are vertically integrated and inseparable within the device.

Question 2: Logically Centralized Control (*SDN*)

What is meant by a control plane that is based on **logically centralized control**? In such cases, are the data plane and the control plane implemented within the same device or in separate devices? Explain.

Solution

Logically centralized control (often associated with SDN) means:

- A **logically central controller** computes and distributes forwarding tables to all routers.
- Individual routers **do not compute** their own tables; they simply execute the instructions provided by the controller.

Implementation: The planes are implemented in **separate devices**:

- **Control Plane:** Implemented in a central server or cluster of servers.
- **Data Plane:** Implemented in the hardware of each individual router.

2. 5.2 Routing Algorithms

Question 3: Centralized vs. Distributed Algorithms

Compare and contrast the properties of a **centralized** and a **distributed** routing algorithm. Give an example of a routing protocol for each.

Solution

Centralized Routing Algorithm: Computes paths using **global knowledge** of the network. It requires complete information about connectivity and link costs for all nodes.

→ *Example:* **OSPF** (Link-State).

Distributed Routing Algorithm: Calculates paths in an **iterative, decentralized manner**. No single node has complete information; nodes start with knowledge of neighbors and exchange info to gradually build paths.

→ *Example:* **BGP** (Distance-Vector/Path-Vector).

Question 4: Link-State vs. Distance-Vector

Compare and contrast **Link-State (LS)** and **Distance-Vector (DV)** routing algorithms.

Solution

1. Link-State (LS):

- Uses **global network knowledge**.
- Each node broadcasts its link costs to *all* other nodes.
- All nodes have a complete map of the network and run Dijkstra's algorithm.

2. Distance-Vector (DV):

- **Iterative, asynchronous, and distributed.**
- A node only knows the cost to its neighbors and the reported costs from neighbors to destinations.
- Nodes only exchange information with *directly attached* neighbors.

Question 5: Count-to-Infinity Problem

Explain the **count-to-infinity problem** in distance vector routing using a suitable example.

Solution

The **count-to-infinity problem** is a limitation of **distance vector routing algorithms** in which routers take a very long time to converge after a link cost increases or a link fails. During this time, routers repeatedly exchange incorrect routing information and gradually increase the distance metric toward infinity.

Network Topology: Consider a network with three routers X , Y , and Z having the following link costs:

- $c(X, Y) = 4$
- $c(Y, Z) = 1$
- $c(X, Z) = 50$

Step 1: Initial Stable State

- Router Y reaches destination X directly with cost 4
- Router Z reaches X via Y with total cost 5 ($1 + 4$)
- All routing tables are correct and the network is stable

Step 2: Link Cost Increase Assume the cost of link (X, Y) increases from 4 to 60.

- Router Y immediately updates its distance to X as 60

Step 3: First Incorrect Update

- Router Z has not yet learned about the link cost increase
- Z still believes that Y has a valid path to X
- Z updates its distance to X as $1 + 60 = 61$

Step 4: Second Incorrect Update

- Router Y receives the updated distance from Z
- Y incorrectly assumes that Z has a better route to X
- Y updates its distance to X as $1 + 61 = 62$

Step 5: Counting to Infinity

- Router Z receives the new value from Y and updates its distance to 63
- Router Y then updates its distance to 64
- This process continues, with both routers increasing their distance values one step at a time

Step 6: Final Convergence The distance metric continues increasing until:

- The metric reaches a predefined maximum value (called infinity), or
- Router Z finally selects the direct route $Z \rightarrow X$ with cost 50

Conclusion: This slow, step-by-step increase of routing costs caused by routers repeatedly advertising incorrect paths is known as the **count-to-infinity problem**. It occurs because distance vector routers lack global network knowledge and cannot immediately detect routing loops.

Question 6: Graph Models and Path Enumeration

Looking at the graph model of a computer network in **Figure 5.3**, enumerate all simple paths (paths without loops) from node **y** to node **u**.

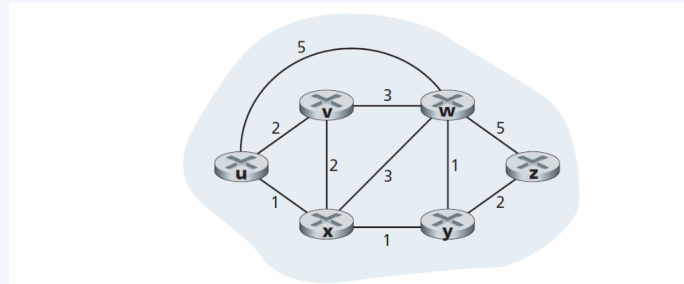


Figure 5.3 ♦ Abstract graph model of a computer network

Solution

The simple paths (those containing no loops) from node **y** to node **u** are:

- $y - x - u$
- $y - x - v - u$
- $y - x - w - u$
- $y - x - w - v - u$
- $y - w - u$
- $y - w - v - u$
- $y - w - x - u$
- $y - w - x - v - u$
- $y - w - v - x - u$
- $y - z - w - u$
- $y - z - w - v - u$
- $y - z - w - x - u$
- $y - z - w - x - v - u$
- $y - z - w - v - x - u$

Question 7: Further Path Enumerations

Repeat the previous problem for all simple paths (those without loops) from node **x** to **z**, node **z** to **u**, and node **z** to **w**.

Solution

Based on the graph in **Figure 5.3**, the simple paths are:

1. Paths from x to z:

- $x - y - z$
- $x - y - w - z$
- $x - w - z$
- $x - w - y - z$
- $x - v - w - z$
- $x - v - w - y - z$
- $x - u - w - z$
- $x - u - w - y - z$
- $x - u - v - w - z$
- $x - u - v - w - y - z$

2. Paths from z to u:

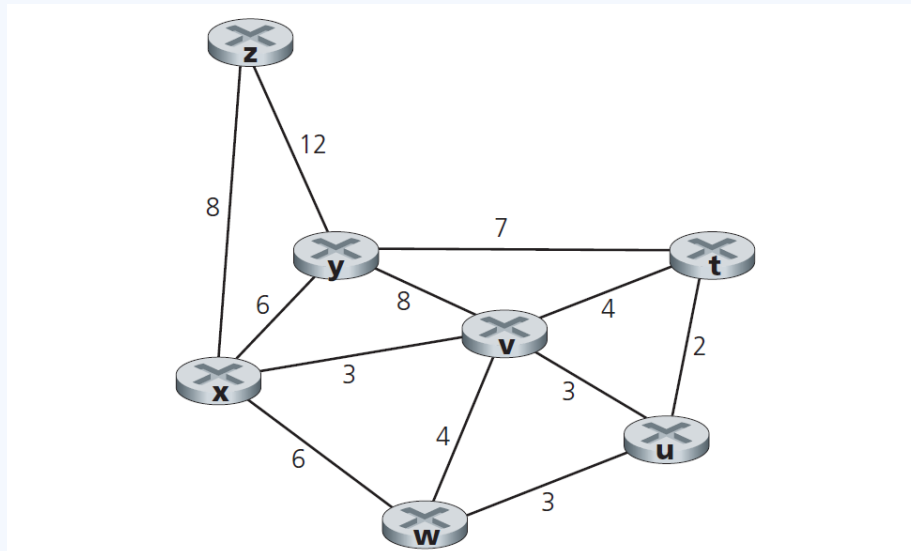
- $z - w - u$
- $z - w - v - u$
- $z - w - x - u$
- $z - w - v - x - u$
- $z - w - x - v - u$
- $z - w - y - x - u$
- $z - w - y - x - v - u$
- $z - y - x - u$
- $z - y - x - v - u$
- $z - y - x - w - u$
- $z - y - x - v - w - u$
- $z - y - w - v - u$
- $z - y - w - x - u$
- $z - y - w - v - x - u$
- $z - y - w - x - v - u$

3. Paths from z to w:

- $z - w$
- $z - y - w$
- $z - y - x - w$
- $z - y - x - v - w$
- $z - y - x - u - w$
- $z - y - x - u - v - w$
- $z - y - x - v - u - w$

Question 8: Dijkstra's Shortest-Path Algorithm

Consider the network graph shown in below. With the indicated link costs, use **Dijkstra's shortest-path algorithm** to compute the shortest path from node **x** to all other network nodes. Show how the algorithm works by providing a step-by-step table.



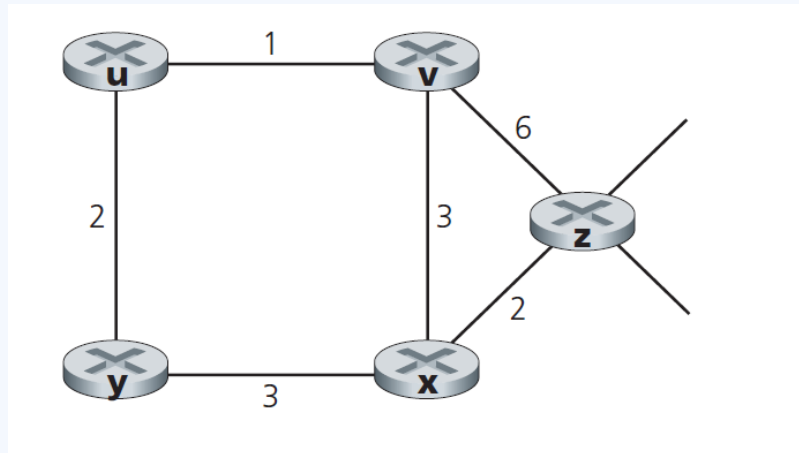
Solution

The following table shows the execution of Dijkstra's algorithm starting from node **x**:

Step	N'	D(t),p(t)	D(u),p(u)	D(v),p(v)	D(w),p(w)	D(y),p(y)	D(z),p(z)
0	x	∞	∞	3,x	6,x	6,x	8,x
1	xv	7,v	6,v	3,x	6,x	6,x	8,x
2	xvu	7,v	6,v	3,x	6,x	6,x	8,x
3	xvuw	7,v	6,v	3,x	6,x	6,x	8,x
4	xvuwy	7,v	6,v	3,x	6,x	6,x	8,x
5	xvuwyt	7,v	6,v	3,x	6,x	6,x	8,x
6	xvuwytz	7,v	6,v	3,x	6,x	6,x	8,x

Question 9: Distance-Vector Routing: Node z

Consider the network shown in the following image. Assume that each node initially knows the costs to each of its neighbors. Using the **distance-vector algorithm**, show the distance table entries at node **z** for each iteration until convergence.



Solution

The following tables show the evolution of the distance table at node **z**. In each table, the rows represent the reported distances from **z**'s neighbors (**v** and **x**), and the last row represents **z**'s own distance vector.

Iteration 0: Initial State

Cost to	u	v	x	y	z
v	∞	∞	∞	∞	∞
From x	∞	∞	∞	∞	∞
z	∞	6	2	∞	0

Iteration 1:

Cost to	u	v	x	y	z
v	1	0	3	∞	6
From x	∞	3	0	3	2
z	7	5	2	5	0

Iteration 2:

Cost to	u	v	x	y	z
v	1	0	3	3	5
From x	4	3	0	3	2
z	6	5	2	5	0

Iteration 3 (Converged):

Cost to	u	v	x	y	z
v	1	0	3	3	5
From x	4	3	0	3	2
z	6	5	2	5	0

Question 10: DV Algorithm: Convergence and Network Diameter

Consider a general topology and a synchronous version of the **distance-vector algorithm**. Suppose that at each iteration, a node exchanges its distance vectors with its neighbors and receives their distance vectors. Assuming that the algorithm begins with each node knowing only the costs to its immediate neighbors, what is the **maximum number of iterations** required before the distributed algorithm converges? Justify your answer.

Solution

In a synchronous execution of the distance-vector algorithm:

1. Reasoning: At each iteration, a node exchanges its distance vector with its neighbors. If node A is a neighbor of B , then after one iteration, all of B 's neighbors (which are at most two hops from A) will know the shortest path cost of one or two hops to A , as reported by B .

2. Network Diameter and Convergence: Let d be the **diameter** of the network—defined as the length of the longest simple path (without loops) between any two nodes.

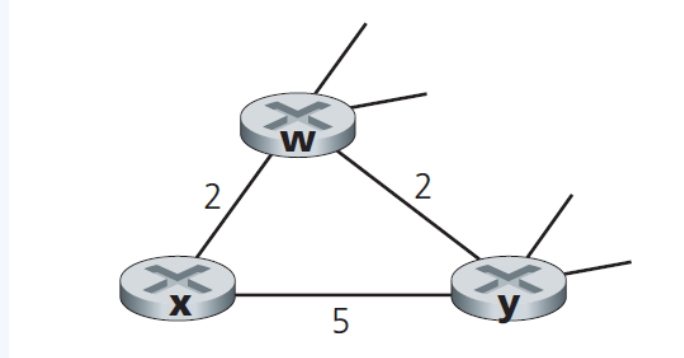
- After d iterations, all nodes will have learned the shortest path costs of d or fewer hops to every other node in the network.
- Since any path with more than d hops must contain loops (making it more costly than the corresponding path with loops removed), d hops are sufficient to cover all possible shortest paths.

Conclusion: Therefore, the algorithm will converge in **at most d iterations**.

Note: If the algorithm is triggered by a link cost change, there is no a priori bound on iterations unless bounds on link costs are specified (due to potential pathologies like count-to-infinity).

Question 11: DV Routing: Link-Cost Changes and Notifications

Consider the network fragment shown below. Node **x** has only two attached neighbors, **w** and **y**. Node **w** has a minimum-cost path to destination **u** (not shown) of 5, and **y** has a minimum-cost path to **u** of 6. The complete paths from **w** and **y** to **u** (and between **w** and **y**) are not shown. All link costs have strictly positive integer values.



- Give node **x**'s distance vector for destinations **w**, **y**, and **u**.
- Give a link-cost change for either $c(x, w)$ or $c(x, y)$ such that **x** will inform its neighbors of a new minimum-cost path to **u** as a result of the distance-vector algorithm.
- Give a link-cost change for either $c(x, w)$ or $c(x, y)$ such that **x** will **not** inform its neighbors of a new minimum-cost path to **u**.

Solution

Based on the values provided ($c(x, w) = 2$, $c(x, y) = 5$, $c(w, y) = 2$, $D_w(u) = 5$, $D_y(u) = 6$):

a) **x**'s distance vector:

- $D_x(w) = 2$ (direct link)
- $D_x(y) = \min(c(x, y), c(x, w) + D_w(y)) = \min(5, 2 + 2) = 4$
- $D_x(u) = \min(c(x, w) + D_w(u), c(x, y) + D_y(u)) = \min(2 + 5, 5 + 6) = 7$

b) Change leading to notification:

- Case 1 (Decrease):** If $c(x, w)$ decreases to 1, the cost to **u** becomes $1 + 5 = 6$. Since the cost changed, **x** informs its neighbors.
- Case 2 (Increase):** If $c(x, w)$ increases to $d > 6$ (e.g., $d = 7$), the new shortest path to **u** will be via **y** ($5 + 6 = 11$). Since the cost changed from 7 to 11, **x** informs its neighbors.

c) Change resulting in no notification:

- Any change in $c(x, y)$ as long as $c(x, y) \geq 1$ will not change $D_x(u)$.
- For example, if $c(x, y)$ changes from 5 to 1, $D_x(u) = \min(2 + 5, 1 + 6) = 7$. Since the minimum cost remains 7, **x**'s distance vector for **u** does not change, and it will not inform its neighbors of a new path to **u**.

Question 12: Count-to-Infinity: Decreasing Link Costs

Consider the **count-to-infinity problem** in distance-vector routing.

- a. Will the count-to-infinity problem occur if we **decrease** the cost of a link? Why?
- b. How about if we connect two nodes which do **not** have a current link?

Solution

No, in both cases, the count-to-infinity problem will not occur.

Reasoning:

- **Decreasing Link Cost:** The count-to-infinity problem is typically caused by positive news (decreased cost) being masked by old, stale information that forms a routing loop. However, simply decreasing a link's cost results in "good news" that propagates quickly. It does not create loops because the decrease in cost makes the existing path more attractive or creates a new, better path, but it doesn't cause nodes to rely on each other in a circular next-hop relationship based on incorrect high-cost estimates.
- **Connecting Two Nodes:** This is mathematically equivalent to **decreasing** a link's cost from infinity to a finite value. As with any other cost decrease, this "good news" propagates through the network without causing routing pathologies like count-to-infinity.

Question 13: DV Algorithm: Monotonicity and Stabilization

Argue that for the distance-vector algorithm (in the context of the previous problem or similar "good news" scenarios), each value in the distance vector $D(x)$ is **non-increasing** and will eventually stabilize in a **finite number of steps**.

Solution

The argument for the monotonicity and stabilization of the distance-vector algorithm is as follows:

1. Non-increasing Property: At each step of the algorithm, the updating of a node's distance vector is governed by the **Bellman-Ford equation**:

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

Because the update rule only accepts a new value if it is smaller than the current one, the values in the distance vector can **only decrease** (or remain the same). There is no mechanism in this basic iteration for values to increase. If no value in the vector is updated, no messages are sent to neighbors. Thus, $D(x)$ is monotonically non-increasing.

2. Finite Stabilization: Since the link costs in the network are finite and strictly positive, there is a lower bound on the cost of any path. Because the distance values are non-increasing and bounded from below by the actual shortest path costs (which are finite), the distance vectors must eventually **stabilize** and stop changing. Once the shortest paths are found, no further updates occur, and the algorithm terminates in a finite number of iterations.

Question 14: Poisoned Reverse and Multi-Node Loops

Suppose there is a network with routers **x**, **y**, **z**, and **w**. Router **w** is connected to routers **y** and **z**. The link costs are: $c(x, y) = 4$, $c(x, z) = 50$, $c(y, w) = 1$, $c(z, w) = 1$, and $c(y, z) = 3$. Suppose that **poisoned reverse** is used.

- When the distance vector routing is stabilized, what distance values do **w**, **y**, and **z** inform their distances to **x** to each other?
- Now suppose that $c(x, y)$ increases to 60. Will there be a **count-to-infinity problem** even if poisoned reverse is used? Why or why not? If so, how many iterations are needed to reach a stable state again?
- How do you modify $c(y, z)$ such that there is no count-to-infinity problem at all if $c(x, y)$ changes from 4 to 60?

Solution

a) **Stabilized Advertisements:** In the stable state, the shortest paths to x are:

- $D_y(x) = 4$ (Direct: $y \rightarrow x$)
- $D_w(x) = 5$ (Path: $w \rightarrow y \rightarrow x$)
- $D_z(x) = 6$ (Path: $z \rightarrow w \rightarrow y \rightarrow x$)

Using **Poisoned Reverse** (if A routes via B , A tells B that $D_A(x) = \infty$):

- **w tells y:** $D_w(x) = \infty$ (routes via y).
- **w tells z:** $D_w(x) = 5$ (routes via y , not z).
- **y tells w / tells z:** $D_y(x) = 4$ (routes directly to x).
- **z tells w:** $D_z(x) = \infty$ (routes via w).
- **z tells y:** $D_z(x) = 6$ (routes via w , not y).

b) **Count-to-Infinity with Poisoned Reverse:** Yes, a count-to-infinity problem occurs.

- **Reason:** Poisoned reverse only prevents loops between two immediate neighbors. It cannot prevent a loop involving three or more nodes (in this case, $y \rightarrow z \rightarrow w \rightarrow y$).
- **Mechanism:** When $c(x, y)$ becomes 60, y sees ∞ from w (poisoned) but sees $D_z(x) = 6$ from z (who routes via w). y chooses z , forming the loop.
- **Iterations:** The loop cost ($y - z - w$) is $3 + 1 + 1 = 5$ per iteration (note: the provided solution suggests the costs rise by roughly 3 per iteration, but the loop $y - z - w - y$ total cost is 5). The counting breaks when $D_z(x)$ via the loop exceeds the direct cost $c(z, x) = 50$. Starting from 6, it takes approximately $\lceil (50 - 6)/3 \rceil \approx 15$ iterations to stabilize (using the provided logic for step increments).

c) **Modifying $c(y, z)$ to Prevent Loops:** To prevent the loop, we need z to prefer direct routing via y in the initial stable state. Current path via w is $c(z, w) + c(w, y) + c(y, x) = 1 + 1 + 4 = 6$. Modified path via y is $c(z, y) + 4$. We need $c(z, y) + 4 < 6$, so $c(z, y) < 2$. **Answer:** Set $c(y, z) = 1$. With this cost, z routes via y and poisons y ($D_z(x) = \infty$). When $x - y$ fails, y sees ∞ from both neighbors and immediately chooses the direct link (cost 60), avoiding the loop.

3. 5.3 Routing in the Internet: Autonomous Systems

Question 15: Intra-AS Routing Uniformity

Is it necessary that every Autonomous System (AS) use the same intra-AS routing algorithm? Why or why not?

Solution

No, it is not necessary.

Reasoning: Each Autonomous System (AS) has **administrative autonomy** for routing within its own network. This means the administrators of an AS are free to choose and configure any intra-AS routing protocol (such as OSPF, EIGRP, or IS-IS) that best suits their internal infrastructure, performance requirements, and management policies.

Uniformity is only required at the **inter-AS** level (using BGP) to ensure different autonomous systems can communicate with each other.

Question 16: Inter-AS vs. Intra-AS Protocols

Why are different inter-AS and intra-AS protocols used in the Internet?

Solution

The distinction between inter-AS and intra-AS protocols exists due to three main factors:

1. Policy:

- **Inter-AS:** Policy issues dominate. An AS may refuse to carry transit traffic or avoid elective paths through specific neighboring ASs for commercial or security reasons.
- **Intra-AS:** Everything is under a single administrative control, so policy concerns are minimal, allowing simpler coordination.

2. Scale:

- **Inter-AS:** Scalability is critical. BGP must handle routing among hundreds of thousands of networks globally.
- **Intra-AS:** Scalability is less of a concern. If a domain becomes too large, it can be split into multiple smaller ASs.

3. Performance:

- **Inter-AS:** Routing is often **policy-oriented**. A longer or more costly path might be chosen simply because it satisfies policy criteria.
- **Intra-AS:** Focuses primarily on **performance** (optimizing for the shortest or least-cost path) because policy issues are less relevant.

4. 5.4 Intra-AS Routing: OSPF

Question 17: OSPF Link-State Broadcasting

True or False: When an OSPF router sends its link-state information, it is sent only to those nodes that are its directly attached neighbors. Explain.

Solution

False.

Explanation: In OSPF (a link-state protocol), a router **broadcasts** its link-state information to **all other routers** within the Autonomous System (AS) it belongs to, not just to its immediate neighbors.

This global broadcasting is necessary because OSPF requires **every router** to:

- Construct a complete, identical **topological map** of the entire AS.
- Locally run **Dijkstra's shortest-path algorithm** to independently determine the least-cost paths to all other nodes in the system.

Question 18: OSPF Areas and Hierarchical Routing

What is meant by an **area** in an OSPF autonomous system? Why was the concept of an area introduced?

Solution

An **area** in an OSPF autonomous system refers to a set of routers in which each router broadcasts its link-state information to all other routers **within the same set**.

Hierarchical Structure: An OSPF AS can be configured hierarchically into multiple areas:

- Each area runs its own independent OSPF link-state routing algorithm.
- **Area Border Routers (ABRs)** are responsible for routing packets outside the area.

Why areas were introduced: The primary reason is **scalability**. By dividing a large OSPF AS into smaller areas, we can:

- Build a hierarchical routing structure for large-scale networks.
- Limit the scope of link-state broadcasts, reducing CPU and memory overhead on individual routers.
- Contain routing instabilities within a specific area.

5. 5.5 Inter-AS Routing: BGP

Question 19: BGP Terminology

Define and contrast the following terms: **subnet**, **prefix**, and **BGP route**.

Solution

Subnet: A portion of a larger network that does not contain a router. Its boundaries are defined by the router and host interfaces.

Prefix: The network portion of a CIDRized address, written in the form **a.b.c.d/x**. A prefix can cover one or more subnets.

BGP Route: In BGP jargon, a **route** is the combination of a **prefix** and its associated **BGP attributes** (such as AS-PATH and NEXT-HOP) that is advertised across a BGP session.

Question 20: BGP Attributes: NEXT-HOP and AS-PATH

How does BGP use the **NEXT-HOP** attribute? How does it use the **AS-PATH** attribute?

Solution

BGP uses these attributes to manage routing and ensure loop-free paths:

- **AS-PATH:**
 - **Loop Detection:** Routers use this attribute to detect and prevent looping advertisements. If a router sees its own AS ID in the AS-PATH of an incoming advertisement, it rejects the route.
 - **Path Selection:** It is also used to choose among multiple paths to the same prefix (typically preferring the one with the shortest AS-PATH).
- **NEXT-HOP:**
 - Indicates the **IP address** of the first router along an advertised path (outside of the AS receiving the advertisement) to a given prefix.
 - A router uses this attribute when configuring its **forwarding table** to determine which interface to use to reach the external destination.

Question 21: Implementing BGP Routing Policy

Describe how a network administrator of an upper-tier ISP can implement **policy** when configuring BGP.

Solution

BGP policy implementation is primarily achieved through selective route advertisement.

Example Scenario: Consider a Tier-1 ISP **B** that has peering agreements with two other Tier-1 ISPs, **A** and **C**. If ISP **B** does not want to carry transit traffic between **A** and **C**:

- **Action 1:** ISP **B** would **not advertise** to **A** any routes that pass through **C**.
- **Action 2:** ISP **B** would **not advertise** to **C** any routes that pass through **A**.

By withholding these advertisements, **A** and **C** will be unaware of a path through **B**, forcing them to find alternative routes and ensuring **B**'s infrastructure is not used for their mutual transit traffic.

Question 22: BGP Route Advertisement and Policy

True or False: When a BGP router receives an advertised path from its neighbor, it must add its own identity to the received path and then send that new path on to all of its neighbors. Explain.

Solution

False.

Explanation: BGP is a **policy-based** routing protocol. A router is **not required** to append its identity and re-advertise every route it receives.

Scenario for Refusal: A router may choose not to re-advertise a path if it does not wish to serve as a **transit router** for that specific destination. For instance, if the received path points to an external Autonomous System (AS) and the router's AS does not have a commercial or policy-based agreement to carry that traffic, it will simply withhold the advertisement from its other neighbors.

Question 23: BGP: Loop Detection

Describe how loops in paths are detected in the **BGP** protocol.

Solution

Loop detection in BGP is straightforward because of the inclusion of the **AS-PATH** attribute in route advertisements.

Each BGP advertisement includes the complete list of Autonomous Systems (ASes) that a route has passed through to reach a destination. When a BGP peer receives an advertised route, it checks the **AS-PATH** to see if its own AS number is already present.

If its identity is found in the path, the router realizes that accepting this route would result in a **routing loop**. Consequently, it will reject the route and not use it or pass it on to its neighbors.

Question 24: BGP: Route Selection Criteria

Will a BGP router always choose the loop-free route with the **shortest AS-PATH** length? Justify your answer.

Solution

No, the chosen path is not necessarily the one with the shortest AS-PATH.

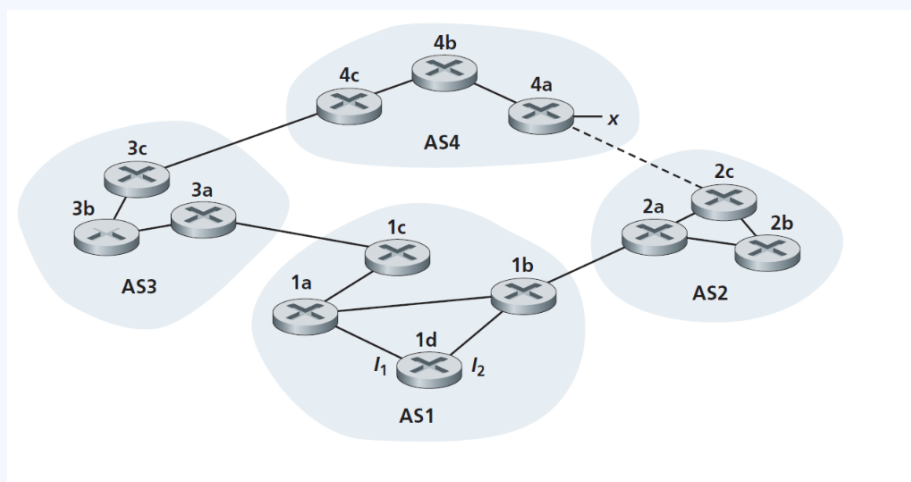
Justification: In BGP, route selection is a multi-step process where **policy and economic reasons** often outweigh simple path length.

- **Commercial Agreements:** An Autonomous System (AS) might prefer sending traffic to a specific neighbor (e.g., a customer providing revenue or a peer with no transit cost) even if the resulting AS-PATH is longer than a route through a provider who charges for transit.
- **Traffic Engineering:** Administrators might prefer a longer, more stable path over a shorter one that is frequently congested.

Therefore, while AS-PATH length is a factor in the selection process, it is only considered after higher-priority policy rules (like Local Preference) are evaluated.

Question 25: Inter-AS and Intra-AS Route Propagation

Consider the network shown below. Suppose AS3 and AS2 are running **OSPF** for their intra-AS routing protocol, while AS1 and AS4 are running **RIP**. **eBGP** and **iBGP** are used for inter-AS routing. Initially, assume there is no physical link between AS2 and AS4.



Determine which routing protocol is used for the following routers to learn about prefix **x** (located in AS4):

- Router **3c** learns about prefix **x** from which protocol?
- Router **3a** learns about **x** from which protocol?
- Router **1c** learns about **x** from which protocol?
- Router **1d** learns about **x** from which protocol?

Solution

Prefix x originates in AS4. The propagation path follows the BGP hierarchy:

- a. **Router 3c:** Learns about x via **eBGP**. 3c is a gateway router for AS3 and receives the advertisement directly from its external peer in AS4 (4c).
- b. **Router 3a:** Learns about x via **iBGP**. As an internal router in AS3, it receives the prefix information distributed by the gateway router (3c) using the internal BGP protocol.
- c. **Router 1c:** Learns about x via **eBGP**. 1c receives the advertisement from its external neighbor in AS3 (3a).
- d. **Router 1d:** Learns about x via **iBGP**. Similar to 3a, it receives the information internally within its own Autonomous System (AS1) from the gateway router (1c).

Question 26: Forwarding Table Selection and Hot-Potato Routing

Referring to the network shown in the previous problem, once router **1d** learns about prefix x , it will put an entry (x , I) in its forwarding table.

- a. Will I be equal to I_1 or I_2 for this entry? Explain why in one sentence.
- b. Now suppose that there is a physical link between AS2 and AS4 (shown by the dotted line). Suppose router **1d** learns that x is accessible via AS2 as well as via AS3. Will I be set to I_1 or I_2 ? Explain why in one sentence.
- c. Now suppose there is another AS, called AS5, which lies on the path between AS2 and AS4 (not shown). Suppose router **1d** learns that x is accessible via AS2–AS5–AS4 as well as via AS3–AS4. Will I be set to I_1 or I_2 ? Explain why in one sentence.

Solution

The selection of the outgoing interface depends on BGP's path selection hierarchy and hot-potato routing:

- a. $I = I_1$: The interface I_1 is selected because it is on the least-cost path from router 1d towards its current gateway router, **1c**, which first advertised the prefix.
- b. $I = I_2$: Since both routes (via AS2 and via AS3) have the same AS-PATH length (length 2), 1d uses **hot-potato routing** and chooses interface I_2 because it leads to the **closest NEXT-HOP router** (1b) that advertised a path to x .
- c. $I = I_1$: In this case, the AS-PATH lengths differ: via AS2 is 3 hops (AS2–AS5–AS4), while via AS3 is 2 hops (AS3–AS4). Since BGP prioritizes the **shortest AS-PATH length** over hot-potato routing, 1d will choose the path via AS3 and set I to I_1 .

6. 5.6 SDN Controller Architecture

Question 27: SDN Controller Layers

Describe the main role of the **communication layer**, the **network-wide state-management layer**, and the **network-control application layer** in an SDN controller.

Solution

A generic SDN controller's functionality is organized into three distinct layers:

- 1. Communication Layer (Southbound Interface):** Responsible for communication between the SDN controller and controlled network devices (switches, hosts, etc.) via a protocol such as **OpenFlow**. It allows the controller to control remote devices and for devices to communicate locally-observed events (e.g., link failures) back to the controller.
- 2. Network-Wide State-Management Layer:** Maintains an accurate, up-to-date view of the **network's state**. This includes information about hosts, links, and switches. It also manages copies of flow tables and statistics, providing the global state required for making informed control decisions.
- 3. Network-Control Application Layer (Northbound Interface):** Provides an API (**Northbound API**) that allows network-control applications to interact with the controller. Through this layer, applications can read/write network state and flow tables, and register to be notified of specific network events.

Question 28: Implementing a New Routing Protocol in SDN

Suppose you wanted to implement a new **routing protocol** in the SDN control plane. At which layer would you implement that protocol? Explain.

Solution

I would implement a new routing protocol at the **SDN's network-control application layer**.

Reasoning: This is the layer where the **“brain”** of the network resides. The network-control application layer is responsible for Determining the end-to-end paths between sources and destinations. By implementing the protocol here, it can leverage the global network state provided by the state-management layer to compute optimal routes and then use the controller's Northbound API to install the resulting rules into the data-plane switches.

Question 29: SDN API Messages and Participants

What types of messages flow across an SDN controller's **northbound** and **southbound** APIs? Who is the recipient of these messages sent from the controller across the southbound interface, and who sends messages to the controller across the northbound interface?

Solution

The SDN controller facilitates communication between applications and network devices through distinct APIs:

1. Southbound API (Controller to Controlled Devices): The recipient of these messages is a **controlled packet switch**. Key message types include:

- **Configuration:** Allows the controller to query and set a switch's configuration parameters.
- **Modify-state:** Used to add, delete, or modify entries in the switch's flow table and set port properties.
- **Read-state:** Used to collect statistics and counter values from the switch's flow table and ports.
- **Send-packet:** Allows the controller to send a specific packet out of a specified port on the switch.

2. Northbound API (Applications to Controller): The senders of these messages are **network-control applications**. These messages typically involve:

- Reading or writing network state.
- Managing flow tables within the controller's state-management layer.

Question 30: OpenFlow Messages

Describe the purpose of **two** types of OpenFlow messages that are sent from a **controlled device to the controller**. Describe the purpose of **two** types of OpenFlow messages that are sent from the **controller to a controlled device**.

Solution

OpenFlow facilitates bidirectional communication between the controller and switches:

1. From Controlled Device to Controller:

- **Flow-removed message:** Informs the controller that a flow table entry has been removed (e.g., due to a timeout or a specific **modify-state** command).
- **Port-status message:** Informs the controller of a change in port status (e.g., a link going down or a port configuration change).

2. From Controller to Controlled Device:

- **Modify-state:** Used to add, delete, or modify entries in the switch's flow table and to configure switch port properties.
- **Read-state:** Used to collect statistics and counter values from the switch's flow table and individual ports.

Question 31: OpenDaylight Service Abstraction Layer (SAL)

What is the purpose of the **service abstraction layer (SAL)** in the OpenDaylight SDN controller?

Solution

The **Service Abstraction Layer (SAL)** in OpenDaylight serves two primary purposes:

- **Internal Communication:** It allows internal network service applications and controller components to communicate with each other. They can invoke each other's services and subscribe to events they generate.
- **Protocol Abstraction:** It provides a **uniform abstract interface** to the specific underlying communications protocols in the communication layer (such as OpenFlow, SNMP, or NETCONF). This allows applications to interact with the data plane without needing to know the specific details of the underlying protocol being used.

7. 5.7 Internet Control Message Protocol (ICMP)

Question 32: ICMP Message Types

Name **four** different types of ICMP messages.

Solution

The following are four common types of ICMP messages along with their type and code values:

- **Echo reply:** Type 0, Code 0 (Used to respond to a ping).
- **Destination network unreachable:** Type 3, Code 0.
- **Destination host unreachable:** Type 3, Code 1.
- **Source quench:** Type 4, Code 0 (Used for basic congestion control).

Question 33: ICMP Messages in Traceroute

What **two** types of ICMP messages are received at the sending host executing the **Traceroute** program?

Solution

While executing Traceroute, the sending host receives the following two types of ICMP messages:

- **ICMP Time Exceeded (Warning) message:** Type 11, Code 0 (Sent by intermediate routers when the TTL reaches zero).
- **Destination Port Unreachable message:** Type 3, Code 3 (Sent by the target host when it receives the Traceroute UDP packet on an uncommon port number).

8. 5.8 Network Management: SNMP

Question 34: SNMP Terminology

Define the following terms in the context of **SNMP**: **managing server**, **managed device**, **network management agent**, and **MIB**.

Solution

The following are key components of the SNMP architecture:

Managing Server: An application, typically residing in a centralized Network Management Station (NMS), that controls the collection, processing, and display of network management information. It is used by administrators to interact with and control network devices.

Managed Device: A piece of network equipment (including its software) that resides on a managed network. Examples include hosts, routers, switches, middleboxes, and even environmental sensors.

Network Management Agent: A software process running on a **managed device** that communicates with the managing server. It performs local actions on the device under the command and control of the server.

Management Information Base (MIB): A repository that collects information associated with managed objects in a network. A MIB object can be a *counter* (e.g., discarded datagrams), *status information* (e.g., device health), or other operational metrics.

Question 35: SNMP Message Types

What are the purposes of the SNMP **GetRequest** and **SetRequest** messages?

Solution

- **GetRequest:** A message sent from a managing server to an agent to request the value of one or more MIB objects at the agent's managed device.
- **SetRequest:** A message used by a managing server to set (modify) the value of one or more MIB objects in a managed device.

Question 36: SNMP Trap Message

What is the purpose of the SNMP **Trap** message?

Solution

An **SNMP trap message** is an unsolicited notification generated by a **managed device** and sent to its managing server in response to a significant event.

Purpose:

- It is used to notify a managing server of an **exceptional situation** (e.g., a link interface going up or down, or a device reboot).
- It alerts the server to changes in MIB object values that require immediate administrative attention, rather than waiting for the server to poll the device.